

Technical Notes

Project: College Decision Platform

Date: 2026-06-08

Architecture Decisions

The project uses a full-stack split between a Next.js frontend and an Express + TypeScript backend. The frontend owns the user experience, routing, filters, compare state, and form interactions. The backend owns validation, database queries, authentication flow, saved colleges, and response consistency.

The backend is layered into controllers, routes, services, models, and utilities. Controllers keep request/response logic thin, services contain database logic, and utilities handle shared concerns such as validation, auth extraction, API responses, and database connection setup.

PostgreSQL is used as the source of truth. Colleges, courses, users, and saved colleges are modeled relationally. Courses are connected to colleges through a join table so the platform can support course-aware search and filtering instead of treating course names as hardcoded UI text.

The API always returns a consistent shape:

```
{
  "success": true,
  "data": {}
}
```

Errors use:

```
{
  "success": false,
  "message": "Error message"
}
```

The frontend does not hardcode college data. It fetches listing data, detail data, saved colleges, compare data, and filter metadata from the backend.

Key Tradeoffs

Authentication is intentionally lightweight for the MVP. A user can login/register with name and email, and the backend creates or returns that user. This is enough to support user-specific saved colleges, but a production version should use password auth, OAuth, or a managed provider such as Clerk, Supabase Auth, or Auth0.

The search implementation is SQL-based and covers college name, city, state, and course names. This is appropriate for the current dataset and MVP scope. At larger scale, the next

step would be PostgreSQL full-text search, trigram indexes, or a search service.

Pagination uses LIMIT/OFFSET because it is simple and clear for an MVP. For much larger datasets or deep pagination, cursor pagination would be more scalable.

The compare system supports 2 to 3 colleges. This keeps the UX readable on desktop and mobile. More colleges would create wide, hard-to-read tables and reduce decision clarity.

The product uses a rule-based decision score rather than machine learning. This keeps recommendations explainable and easy to debug. A later version could add personalized ranking based on user preferences, saved behavior, and exam profile.

Some data fields are nullable, especially placement percentage and average package. The UI displays "Data not available" instead of assuming zero or hiding the field. This avoids misleading students.

What Was Optimized For

The main optimization was clarity. The platform shows the few metrics students need first: rating, fees, placement, average package, ranking, and available courses.

The second optimization was speed. Listing pages use pagination, backend filters are query-driven, filter options are fetched through metadata endpoints, and the frontend avoids redundant hardcoded data.

The third optimization was decision support. The app supports search, filtering, detail view, compare, save, and login-backed shortlist flow. The compare page highlights stronger values so users can make decisions faster.

The fourth optimization was maintainability. The codebase is split into reusable frontend components and backend service layers, so new entities such as exams, reviews, rankings, and predictors can be added without rewriting the entire app.

Deployment Notes

Frontend target: Vercel

Backend target: Render

Database target: Hosted PostgreSQL through Supabase

Production environment variables:

Frontend:

NEXT_PUBLIC_API_URL=https://college-iku0.onrender.com

Backend:

DATABASE_URL=postgresql://...

FRONTEND_URL=https://college-frontend-git-main-maruthichethan9-6326s-projects.vercel.app
PORT=4000

The backend exposes '/health' for deployment verification. The expected response is:

```
{  
  "success": true,  
  "data": {  
    "status": "ok"  
  }  
}
```

Known Limitations and Next Steps

The MVP does not yet include exam pages, review submission, full predictors, or article/content pages. These were included in the research model so the system can grow in that direction.

The current database contains sample demo data. A production version should add verified data ingestion, source tracking, update timestamps, and confidence labels for fees and placements.

The current auth flow is suitable for assignment demonstration, but production auth should be hardened before real users are onboarded.

The current compare score is rule-based. The next version should allow users to weight affordability, placement, location, and course preference.